

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Инженерия Искусственного Интеллекта

Лабораторная работа № 2

Дисциплина: Фронтенд разработка

“Взаимодействие с внешним API и авторизация”

Выполнил:

Фиолетов Эдуард Анджеевич

Группа J3213

Проверил:

Добряков Д. Ю.

Санкт-Петербург

2026 г.

Содержание

1. Цель работы	3
2. Введение и обоснование выбранного стека	3
3. Задание	3
4. Архитектура API	4
5. Авторизация	4
6. Клиентский API-слой	5
7. Получение данных	6
8. Поиск заказов	6
9. Изменение данных	7
10. Интеграция с внешними источниками	7
11. Проверка результата	7
12. Вывод	8

1. Цель работы

Цель лабораторной работы - подключить ранее разработанный интерфейс к API, реализовать получение и изменение данных через HTTP-запросы, добавить авторизацию и обработку защищенных разделов приложения.

Marketplace Alarm Bot 2 - система мониторинга заказов маркетплейсов. Для проекта реализован собственный JSON API на Go. API хранит доменные сущности, проверяет доступы, агрегирует данные из маркетплейсов и обслуживает интерфейс менеджера.

2. Введение и обоснование выбранного стека

В проекте используется связка React + TypeScript + TanStack Query на клиенте и Go net/http на сервере. Такой стек выбран из-за количества асинхронных данных: заказы, города доставки, склады, товары, статистика, ключи доступа и состояние интеграций должны обновляться без перезагрузки страницы.

На клиенте используется обертка `apiFetch<T>` поверх стандартного `fetch`. Она типизирует ответы, централизует обработку HTTP-ошибок и обеспечивает единый формат запросов. TanStack Query используется для кэширования, фонового обновления и инвалидации данных после мутаций. На сервере Go API реализует авторизацию по ключу доступа и защищенные маршруты.

3. Задание

По заданию лабораторной работы необходимо:

- связать интерфейс из первой лабораторной работы с API;
- получать данные через HTTP-запросы;
- реализовать закрытую часть API с авторизацией;
- использовать JSON как основной формат обмена данными;
- вывести данные API в интерфейс;
- обработать пользовательские действия, которые изменяют данные на сервере.

В проекте эти требования реализованы на следующих сценариях:

- вход по ключу доступа;
- загрузка сводки по заказам и городам;
- просмотр заказов по маркетплейсам;
- поиск заказов;
- подтверждение даты доставки;
- переключение признака пригорода;
- управление складами;
- просмотр состояния интеграций.

4. Архитектура API

Регистрация маршрутов API находится в файле `./web/server.go`. Сервер создает HTTP-мультиплексор, регистрирует публичный маршрут входа и защищенные маршруты для данных приложения.

- `./web/server.go` - регистрация JSON API и SPA fallback
- `./web/handlers_auth.go` - логин, текущий пользователь и выход
- `./web/middleware.go` - авторизация, роли, gzip и rate limit
- `./web/data.go` - структуры данных для ответов API

Основные группы API:

- `/api/login`, `/api/logout`, `/api/me` - авторизация и текущий пользователь;
- `/api/nav`, `/api/index` - данные для навигации и главной страницы;
- `/api/orders/{marketplace}` - заказы конкретного маркетплейса;
- `/api/archive` - архив заказов;
- `/api/delivery/{city}` - карточки доставки по городу;
- `/api/search` - поиск заказов;
- `/api/stats`, `/api/stats/{city}` - статистика;
- `/api/products` - товары и комплекты;
- `/api/warehouses` - склады;
- `/api/admin/keys`, `/api/admin/settings` - административные данные.

Фрагмент регистрации маршрутов:

```
mux.HandleFunc("POST /api/login", loginRateLimitMiddleware(s.apiLogin))
mux.HandleFunc("GET /api/me", apiAuth(s.apiMe))
mux.HandleFunc("GET /api/index", apiAuth(s.apiIndex))
mux.HandleFunc("GET /api/delivery/{city}", apiAuth(s.apiDelivery))
mux.HandleFunc("GET /api/orders/{marketplace}", apiAuth(managerUp(s.apiOrders)))
mux.HandleFunc("GET /api/search", apiAuth(managerUp(s.apiSearch)))
```

5. Авторизация

Авторизация реализована по ключу доступа. Пользователь вводит ключ на странице входа, клиент отправляет `POST /api/login`, после чего сервер проверяет ключ и устанавливает cookie `mab_session`.

```
http.SetCookie(w, &http.Cookie{
    Name:    cookieName,
    Value:   body.Key,
    Path:    "/",
    HttpOnly: true,
    Secure:   true,
    SameSite: http.SameSiteStrictMode,
    MaxAge:   90 * 24 * 3600,
})
```

Для защиты API используется middleware `apiAuthMiddleware`. Он извлекает ключ из cookie, проверяет его наличие в конфигурации или базе данных и добавляет найденный ключ в контекст запроса.

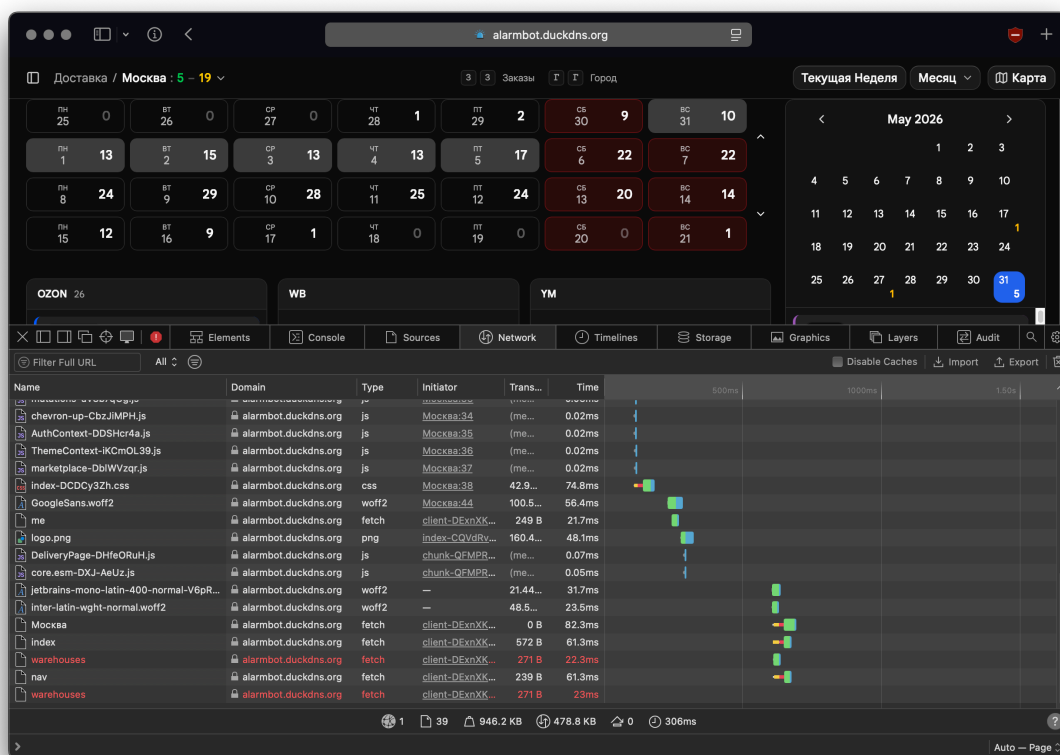
Для разграничения прав используется `requireAccess`. В проекте есть три уровня:

- `full` - полный доступ к административным действиям;
- `manager` - доступ к заказам, доставке, складам и статистике;
- `city` - доступ только к доставке выбранного города.

6. Клиентский API-слой

Клиентский слой находится в каталоге `src/api`. Базовая функция `apiFetch<T>` формирует запрос к `/api`, передает JSON-заголовки, проверяет статус ответа и возвращает типизированный результат.

- `./web/frontend/src/api/client.ts` - унифицированный HTTP-клиент
- `./web/frontend/src/api/queries.ts` - хуки загрузки данных
- `./web/frontend/src/api/mutations.ts` - мутации и инвалидация кэша
- `./web/frontend/src/api/types.ts` - TypeScript-типы ответов API



Проверка API в DevTools Network: запросы к серверу приложения и загрузка данных страницы доставки.

```
export async function apiFetch<T>(path: string, init?: RequestInit): Promise<T> {
  const res = await fetch(`${BASE}${path}`, {
    ...init,
    headers: {
      'Content-Type': 'application/json',
      ...init?.headers,
    },
  });
};
```

```

if (res.status === 401) {
  throw new ApiError(401, 'Unauthorized');
}

if (!res.ok) {
  const body = await res.text();
  throw new ApiError(res.status, body);
}

if (res.status === 204) return null as T;
return res.json();
}

```

Этот фрагмент является единой точкой HTTP-обмена на клиенте. Компоненты и хуки не работают с `fetch` напрямую: они вызывают `apiFetch`, получают типизированный JSON-ответ и одинаковую обработку ошибок авторизации.

Типы в `src/api/types.ts` повторяют структуру серверных ответов: `AuthInfo`, `IndexData`, `OrdersData`, `DeliveryData`, `SearchData`, `WarehousesData` и другие. Они помогают проверять соответствие данных еще на этапе разработки.

7. Получение данных

Для загрузки данных используется `TanStack Query`. Каждая функция описывает отдельный запрос и его ключ кэша.

```

export function useOrders(marketplace: string) {
  return useQuery({
    queryKey: ['orders', marketplace],
    queryFn: () => apiFetch<OrdersData>(`/orders/${marketplace}`),
    refetchInterval: POLL_INTERVAL,
  });
}

```

Примеры запросов:

- `useAuth()` получает текущего пользователя через `/api/me`;
- `useIndex()` загружает главную сводку через `/api/index`;
- `useOrders(marketplace)` загружает заказы маркетплейса;
- `useDelivery(city)` загружает карточки доставки по городу;
- `useSearch(query)` выполняет поиск заказов;
- `useWarehouses()` получает список складов.

Для живых разделов задан интервал фонового обновления `60_000` миллисекунд. Так интерфейс получает новые заказы без ручного обновления страницы.

8. Поиск заказов

Поиск реализован маршрутом `/api/search?q=...`. Сервер нормализует строку поиска, разбивает ее на токены, отдельно выделяет цифры из телефонных номеров и передает фильтры в репозиторий.

```

q := strings.TrimSpace(r.URL.Query().Get("q"))
if len(q) < 2 {
    jsonOK(w, map[string]any{"orders": []sqlite.SearchResultRow{}})
    return
}
tokens := strings.Fields(strings.ToLower(q))
results, err := s.repo.SearchOrders(r.Context(), filters)

```

На клиенте поиск встроен в командную палитру. При вводе двух и более символов вызывается `useSearch`, после выбора результата пользователь попадает на страницу доставки нужного города и сразу видит найденный заказ.

9. Изменение данных

Изменение данных реализовано через мутации. Например, подтверждение доставки отправляет `POST /api/delivery/{city}/confirm`. После отправки запросов кэш доставки инвалидируется, чтобы интерфейс получил актуальное состояние.

```

export function useConfirmDelivery(city: string) {
    return useMutation({
        mutationFn: ({ externalId, marketplace, date }) =>
            apiFetch(`/delivery/${encodeURIComponent(city)}/confirm`, {
                method: 'POST',
                body: JSON.stringify({ externalId, marketplace, date }),
            }),
        onSettled: () => {
            qc.invalidateQueries({ queryKey: ['delivery', city] });
        },
    });
}

```

В интерфейсе это используется на странице доставки: пользователь выбирает дату, переносит карточку или нажимает действие подтверждения, после чего карточка отображается в группе подтвержденных заказов.

10. Интеграция с внешними источниками

Marketplace Alarm Bot 2 получает исходные данные из внешних API маркетплейсов. В веб-слое эти данные представлены как единый JSON API для интерфейса. Такой слой скрывает различия между маркетплейсами и отдает клиенту нормализованные сущности: заказ, карточка доставки, товар, склад, город, статус.

За счет этого интерфейс не зависит от формата каждого конкретного маркетплейса. Например, список заказов Wildberries, Ozon и Яндекс Маркета отображается через одну таблицу, а доставка по городам собирается в единую доску.

11. Проверка результата

Проверка лабораторной работы выполнялась по следующим сценариям:

1. Ввести ключ доступа на странице входа.
2. Проверить запрос `POST /api/login` и установку `cookie`.
3. Открыть главную страницу и проверить запрос `/api/index`.

4. Открыть заказы Wildberries, Ozon и Яндекс Маркета.
5. Выполнить поиск заказа через командную палитру.
6. Открыть страницу доставки города и подтвердить дату доставки.
7. Проверить, что после мутации данные страницы обновляются.
8. Проверить отказ в доступе при недостаточном уровне прав.

12. Вывод

В лабораторной работе интерфейс Marketplace Alarm Bot 2 подключен к JSON API. Реализованы авторизация по ключу, защищенные маршруты, загрузка данных заказов, поиск, обновление состояния доставки и работа с ролями. Клиентский слой использует типизированные запросы и кэширование, а серверный слой предоставляет единый API поверх данных маркетплейсов. Реализация покрывает требования работы по взаимодействию с внешним API и закрытыми разделами приложения.